



Task 3: Model Deployment

Uber Fare Prediction — Regression Model Deployment with Flask

Overview

In Task 1 and Task 2 you preprocessed the Uber fare dataset and trained several regression models to predict the fare amount. In this task, you will compare the models you trained, select the best-performing one, and deploy it as a web service using Flask so that it can accept new trip details and return a predicted fare in real time.

Learning Objectives

- Compare multiple regression models using appropriate evaluation metrics.
- Justify the selection of a final model based on evidence, not assumption.
- Persist a trained model and its preprocessing steps for reuse outside the notebook.
- Build a functioning Flask API that serves predictions from the saved model.
- Test the deployed endpoint with real request/response examples.

Dataset

Continue using the Uber fare dataset from the previous tasks (the same cleaned/preprocessed version you produced in Task 1). The target variable is **fare_amount**. Feature columns should match whatever you engineered previously (e.g. trip distance, pickup/dropoff coordinates or derived distance, passenger count, and any time-based features such as hour of day or day of week).

Part A — Model Comparison

Using your preprocessed training and test sets, train and evaluate at least three regression models, for example:

- Linear Regression
 - Random Forest Regressor
 - Decision Tree Regressor or Gradient Boosting Regressor (optional third/fourth model)
- For each model, report the following metrics on the test set:

Metric	What it tells you
MAE (Mean Absolute Error)	Average absolute difference between predicted and actual fare
RMSE (Root Mean Squared Error)	Penalizes larger errors more heavily than MAE
R ² Score	Proportion of variance in fare explained by the model

Summarize your results in a comparison table in your notebook, and write 3-5 sentences explaining which model you are choosing to deploy and why (do not choose the model with the lowest training error alone — base your decision on the test-set metrics above, and consider prediction speed and interpretability if the metrics are close).

Part B — Save the Model

Once you have selected your best model, retrain it on the full training data (if needed) and save it to disk along with any preprocessing objects (e.g. a scaler or encoder) so that raw input can be transformed the same way at prediction time.

Example:

```
import joblib
joblib.dump(best_model, 'uber_fare_model.pkl')
joblib.dump(scaler, 'uber_fare_scaler.pkl') # if you used one
```

Part C — Build the Flask App (UI + Backend)

Build a Flask web application with a simple, usable interface that lets a user enter trip details and get back a predicted fare. This part is about designing and building the application and the pipeline behind it yourselves, using what you already know from Task 2 — no starter code is provided.

Your app must include:

- **A web page (HTML form)** where the user can enter the trip inputs your model needs (e.g. trip distance, pickup/dropoff location or coordinates, passenger count, date/time of the trip). Label each field clearly and use sensible input types (number fields for numeric values, a date/time picker for the trip time, etc.).
- **A results view** that displays the predicted fare back to the user in a clear, readable way after they submit the form.
- **Basic input handling on the page itself**, such as marking required fields and giving sensible defaults, so the user cannot easily submit an empty or malformed form.
- **The exact same data pipeline used in Task 2** applied to whatever the user submits — the raw form values must go through the same cleaning, feature engineering, and scaling/encoding steps you used before training, in the same order, before being handed to the model. If the user's raw input skips this pipeline, the prediction will be wrong even though the app runs without errors.
- **Reasonable handling of bad or missing user input** — if a required field is empty, a number is out of a sensible range, or a value cannot be parsed, the app should show the user a clear message instead of crashing or silently producing a prediction.
- The trained model and any preprocessing objects from Part B loaded once when the app starts, not retrained on every request.

You are free to choose the structure of the app (single page vs. separate routes for the form and the result, plain HTML/CSS vs. a template engine, etc.), as long as the pipeline requirement above is respected end-to-end.

Part D — Test the App

Run your app locally and test it by entering at least three different example trips through the actual UI (not by calling the backend directly). For each test, include a screenshot of the filled-in form and the resulting prediction shown on the page. Also test at least one case with missing or invalid input and show that the app responds with a clear message rather than an error page or a silent wrong prediction.

Deliverables

- Notebook or script showing training and comparison of all models with metrics table.
- Short written justification (3-5 sentences) of the chosen model.
- Saved model file(s) (.pkl or equivalent).
- The Flask app (UI + backend) implementing the Task 2 pipeline end-to-end for user-submitted data.
- Screenshots of at least 3 successful predictions through the UI, plus 1 invalid-input case.

Grading Rubric

Criteria	Points
Model comparison (metrics reported correctly for all models)	25
Justification of chosen model based on evidence	15
Model + preprocessing objects saved correctly	10
UI form (clear fields, required-field handling, readable result view)	20
Correct application of the Task 2 pipeline to user-submitted data	20
Handling of missing/invalid user input	10
Testing evidence (3+ successful predictions + 1 invalid-input case)	10

Note: Do not skip preprocessing at prediction time. A common mistake is training on scaled/encoded features but sending raw values to the model in the Flask app — this will produce badly wrong predictions even though the API runs without errors. Always test with values you can sanity-check by hand.

Submission: Submit your notebook, app.py, saved model file(s), and a short README describing how to run the Flask app locally.

